Hướng Dẫn Triển Khai Kubernetes Dự Án Pharmacy (3 Node - VirtualBox)

Tài liệu này tổng hợp toàn bộ quy trình chuẩn bị hạ tầng, cấu hình máy ảo, dựng cụm Kubernetes (1 Master + 2 Worker) và triển khai ứng dụng Pharmacy Project từ A-Z.

💻 1. Thông Tin Kiến Trúc & Cấu Hình Máy ẢO (VirtualBox)

Cấu hình phần cứng khuyên dùng (Laptop 16GB RAM / Intel Core i5):

- **Hệ điều hành máy ảo**: **Ubuntu Server 22.04 LTS** (Bắt buộc dùng bản Server CLI để tiết kiệm RAM).
- **Ổ đĩa (VDI)**: Tạo tối thiểu **25 - 30 GB** cho mỗi máy ảo để tránh lỗi đầy đĩa (`disk-pressure`).

Máy ảo (Node)	Tên máy (Hostname)	IP Ví dụ	Số vCPU	RAM	Ổ đĩa	Nhiệm vụ
Master Node	`master-node`	`192.168.1.10`	2 vCPU	2.0 GB	25 GB	Điều phối hệ thống (Control Plane)
Worker Node 1	`worker-node-1`	`192.168.1.11`	2 vCPU	3.5 GB	30 GB	Chạy Frontend + Backend + **MongoDB**
Worker Node 2	`worker-node-2`	`192.168.1.12`	2 vCPU	2.5 GB	25 GB	Chạy Frontend + Backend

---

🛠 2. GIAI ĐOẠN 1: Chuẩn Bị Hạ Tầng (Chạy trên TẤT CẢ 3 Máy Ảo)

Thực hiện lần lượt các lệnh sau trên cả **Master Node**, **Worker Node 1** và **Worker Node 2**:

Bước 1.1: Đặt tên Hostname chuẩn

- **Trên Master Node**:sudo hostnamectl set-hostname master-node
  echo "127.0.1.1 master-node" | sudo tee -a /etc/hosts

```
khoavo@ovl-iuh:~$ sudo hostnamectl set-hostname master-node
echo "127.0.1.1 master-node" | sudo tee -a /etc/hosts
127.0.1.1 master-node
khoavo@ovl-iuh:~$
```

- **Trên Worker Node 1**:sudo hostnamectl set-hostname worker-node-1
  echo "127.0.1.1 worker-node-1" | sudo tee -a /etc/hosts

```
khoavo@worker-node-1:~$ sudo hostnamectl set-hostname worker-node-1
echo "127.0.1.1 worker-node-1" | sudo tee -a /etc/hosts
127.0.1.1 worker-node-1
khoavo@worker-node-1:~$
```

- **Trên Worker Node 2**:sudo hostnamectl set-hostname worker-node-2
  echo "127.0.1.1 worker-node-2" | sudo tee -a /etc/hosts

```
khoavo@worker2:~$ sudo hostnamectl set-hostname worker-node-2
echo "127.0.1.1 worker-node-2" | sudo tee -a /etc/hosts
127.0.1.1 worker-node-2
khoavo@worker2:~$
```

Bước 1.2: Tắt Swap (Bắt buộc với K8s)

sudo swapoff -a
sudo sed -i '/ swap / s/^\(.*\)$/#\1/g' /etc/fstab

```
khoavo@ovl-iuh:~$ sudo swapoff -a
sudo sed -i '/ swap / s/^\(.*\)$/#\1/g' /etc/fstab
khoavo@ovl-iuh:~$
```

Bước 1.3: Cấu hình Kernel Modules & Mạng Linux

cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF

sudo modprobe overlay
sudo modprobe br_netfilter

cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward   = 1

EOF

sudo sysctl --system

```
khoavo@ovl-iuh:~$ cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF

sudo modprobe overlay
sudo modprobe br_netfilter

cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables  = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward                 = 1
EOF

sudo sysctl --system
overlay
br_netfilter
net.bridge.bridge-nf-call-iptables  = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward                 = 1
* Applying /etc/sysctl.d/10-console-messages.conf ...
kernel.printk = 4 4 1 7
* Applying /etc/sysctl.d/10-ipv6-privacy.conf ...
net.ipv6.conf.all.use_tempaddr = 2
net.ipv6.conf.default.use_tempaddr = 2
* Applying /etc/sysctl.d/10-kernel-hardening.conf ...
kernel.kptr_restrict = 1
* Applying /etc/sysctl.d/10-magic-sysrq.conf ...
kernel.sysrq = 176
* Applying /etc/sysctl.d/10-network-security.conf ...
net.ipv4.conf.default.rp_filter = 2
net.ipv4.conf.all.rp_filter = 2
* Applying /etc/sysctl.d/10-ptrace.conf ...
kernel.yama.ptrace_scope = 1
* Applying /etc/sysctl.d/10-zeropage.conf ...
vm.mmap_min_addr = 65536
* Applying /usr/lib/sysctl.d/50-default.conf ...
kernel.core_uses_pid = 1
net.ipv4.conf.default.rp_filter = 2
net.ipv4.conf.default.accept_source_route = 0
```

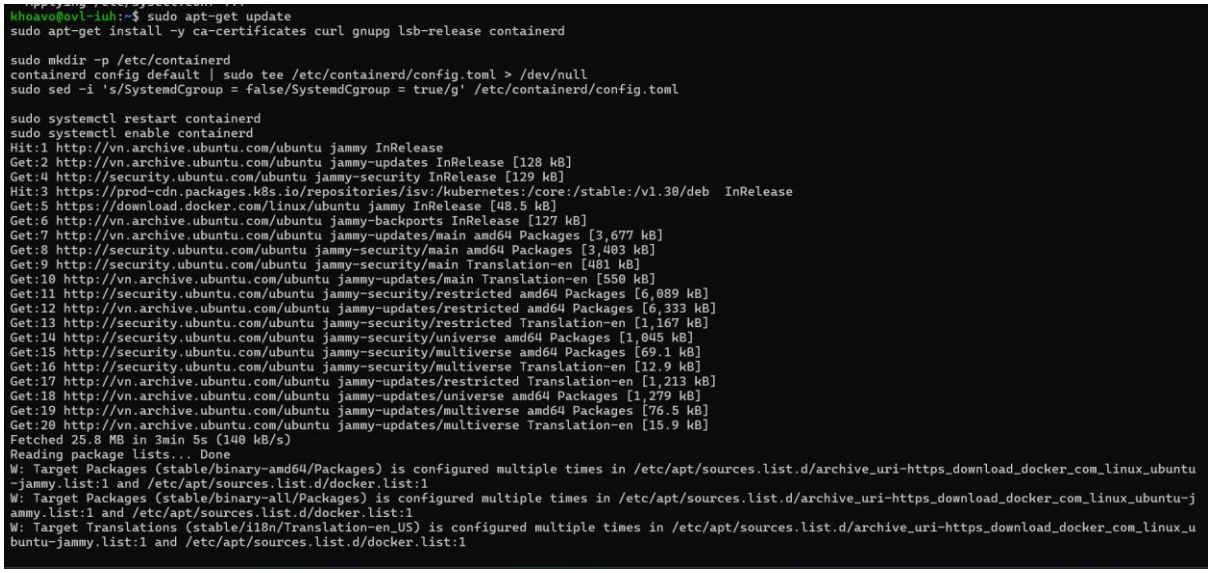
Bước 1.4: Cài đặt Container Runtime (`containerd`)

sudo apt-get update
sudo apt-get install -y ca-certificates curl gnupg lsb-release containerd

sudo mkdir -p /etc/containerd
containerd config default | sudo tee /etc/containerd/config.toml > /dev/null

sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/g' /etc/containerd/config.toml

sudo systemctl restart containerd
sudo systemctl enable containerd



Bước 1.5: Cài đặt Bộ công cụ K8s (`kubelet`, `kubeadm`, `kubectl`)

sudo apt-get update
sudo apt-get install -y apt-transport-https ca-certificates curl gpg
sudo mkdir -p -m 755 /etc/apt/keyrings

curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.30/deb/Release.key | sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg --yes

echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.30/deb/ /' | sudo tee /etc/apt/sources.list.d/kubernetes.list

sudo apt-get update
sudo apt-get install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl
sudo systemctl enable --now kubelet

```
khoavo@ovl-iuh:~$ sudo apt-get update
sudo apt-get install -y apt-transport-https ca-certificates curl gpg
sudo mkdir -p -m 755 /etc/apt/keyrings

curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.30/deb/Release.key | sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg --yes

echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.30/deb/ /' | sudo tee /etc/apt/sources.list.d/kubern
etes.list

sudo apt-get update
sudo apt-get install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl
sudo systemctl enable --now kubelet
Hit:1 http://vn.archive.ubuntu.com/ubuntu jammy InRelease
Hit:2 http://vn.archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:3 http://vn.archive.ubuntu.com/ubuntu jammy-backports InRelease
Hit:4 https://download.docker.com/linux/ubuntu jammy InRelease
Hit:5 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:6 https://prod-cdn.packages.k8s.io/repositories/isv:/kubernetes:/core:/stable:/v1.30/deb  InRelease
Reading package lists... Done
W: Target Packages (stable/binary-amd64/Packages) is configured multiple times in /etc/apt/sources.list.d/archive_uri-https_download_docker_com_linux_ubuntu
-jammy.list:1 and /etc/apt/sources.list.d/docker.list:1
W: Target Packages (stable/binary-all/Packages) is configured multiple times in /etc/apt/sources.list.d/archive_uri-https_download_docker_com_linux_ubuntu-j
ammy.list:1 and /etc/apt/sources.list.d/docker.list:1
W: Target Translations (stable/i18n/Translation-en_US) is configured multiple times in /etc/apt/sources.list.d/archive_uri-https_download_docker_com_linux_u
buntu-jammy.list:1 and /etc/apt/sources.list.d/docker.list:1
W: Target Translations (stable/i18n/Translation-en) is configured multiple times in /etc/apt/sources.list.d/archive_uri-https_download_docker_com_linux_ubun
tu-jammy.list:1 and /etc/apt/sources.list.d/docker.list:1
W: Target CNF (stable/cnf/Commands-amd64) is configured multiple times in /etc/apt/sources.list.d/archive_uri-https_download_docker_com_linux_ubuntu-jammy.l
ist:1 and /etc/apt/sources.list.d/docker.list:1
W: Target CNF (stable/cnf/Commands-all) is configured multiple times in /etc/apt/sources.list.d/archive_uri-https_download_docker_com_linux_ubuntu-jammy.lis
t:1 and /etc/apt/sources.list.d/docker.list:1
W: Target Packages (stable/binary-amd64/Packages) is configured multiple times in /etc/apt/sources.list.d/archive_uri-https_download_docker_com_linux_ubuntu
-jammy.list:1 and /etc/apt/sources.list.d/docker.list:1
W: Target Packages (stable/binary-all/Packages) is configured multiple times in /etc/apt/sources.list.d/archive_uri-https_download_docker_com_linux_ubuntu-j
ammy.list:1 and /etc/apt/sources.list.d/docker.list:1
W: Target Translations (stable/i18n/Translation-en_US) is configured multiple times in /etc/apt/sources.list.d/archive_uri-https_download_docker_com_linux_u
buntu-jammy.list:1 and /etc/apt/sources.list.d/docker.list:1
W: Target Translations (stable/i18n/Translation-en) is configured multiple times in /etc/apt/sources.list.d/archive_uri-https_download_docker_com_linux_ubun
```

🚀 3. GIAI ĐOẠN 2: Khởi Tạo Cluster & Kết Nối Các Node

Bước 2.1: Khởi tạo Master Node (Chỉ chạy trên `master-node`)

1. Lấy IP của máy Master: `hostname -I` (Giả sử IP là `192.168.1.10`).

2. Khởi tạo cụm:sudo kubeadm init --pod-network-cidr=10.244.0.0/16 --apiserver-advertise-address=192.168.1.10

3. Copy và lưu lại dòng lệnh `sudo kubeadm join 192.168.1.10:6443 --token ...` hiển thị ở cuối màn hình.

```
khoavo@ovl-iuh:~$ sudo kubeadm init --pod-network-cidr=10.244.0.0/16 --apiserver-advertise-address=192.168.1.10
I0724 02:52:05.783901    4008 version.go:256] remote version is much newer: v1.36.3; falling back to: stable-1.30
[init] Using Kubernetes version: v1.30.14
[preflight] Running pre-flight checks
[preflight] Pulling images required for setting up a Kubernetes cluster
[preflight] This might take a minute or two, depending on the speed of your internet connection
```

```
Then you can join any number of worker nodes by running the following on each as root:

kubeadm join 192.168.1.10:6443 --token 5s0nq2.8ryscyi4165t6r26 \
        --discovery-token-ca-cert-hash sha256:e0d392bf5839d134ea88f44f7157122a9928d640f59c3f8c48b52d85a5f9088c
khoavo@ovl-iuh:~$
```

Bước 2.2: Cấu hình quyền `kubectl` (Chỉ chạy trên `master-node`)

mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config

sudo chown $(id -u):$(id -g) $HOME/.kube/config

```
khoavo@master-node:~$ mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
[sudo] password for khoavo:
khoavo@master-node:~$
```

Bước 2.3: Cài đặt Mạng Flannel CNI (Chỉ chạy trên `master-node`)

kubectl apply -f https://raw.githubusercontent.com/flannel-io/flannel/master/Documentation/kube-flannel.yml

```
khoavo@master-node:~$ kubectl apply -f https://raw.githubusercontent.com/fla
nnel-io/flannel/master/Documentation/kube-flannel.yml
namespace/kube-flannel created
clusterrole.rbac.authorization.k8s.io/flannel created
clusterrolebinding.rbac.authorization.k8s.io/flannel created
serviceaccount/flannel created
configmap/kube-flannel-cfg created
daemonset.apps/kube-flannel-ds created
```

Bước 2.4: Nối 2 máy Worker Node vào Cluster

Đăng nhập vào **Worker Node 1** và **Worker Node 2**, chạy lệnh `join` đã lưu ở Bước 2.1:

sudo kubeadm join 192.168.1.10:6443 --token <TOKEN> --discovery-token-ca-cert-hash sha256:<HASH>

(Nếu lỡ quên lệnh join, trên máy Master gõ: `kubeadm token create --print-join-command`)

```
khoavo@worker-node-1:~$ sudo kubeadm join 192.168.1.10:6443 --token 5s0nq2.8
ryscyi4165t6r26 \
        --discovery-token-ca-cert-hash sha256:e0d392bf5839d134ea88f44f715712
2a9928d640f59c3f8c48b52d85a5f9088c
[preflight] Running pre-flight checks
[preflight] Reading configuration from the cluster...
[preflight] FYI: You can look at this config file with 'kubectl -n kube-syst
em get cm kubeadm-config -o yaml'
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/conf
ig.yaml"
[kubelet-start] Writing kubelet environment file with flags to file "/var/li
b/kubelet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-check] Waiting for a healthy kubelet at http://127.0.0.1:10248/heal
thz. This can take up to 4m0s
[kubelet-check] The kubelet is healthy after 1.006910434s
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap

This node has joined the cluster:
* Certificate signing request was sent to apiserver and a response was recei
ved.
* The Kubelet was informed of the new secure connection details.

Run 'kubectl get nodes' on the control-plane to see this node join the clust
er.

khoavo@worker-node-1:~$
```

```
khoavo@worker-node-2:~$ sudo kubeadm join 192.168.1.10:6443 --token 5s0nq2.8
ryscyi4165t6r26 \
        --discovery-token-ca-cert-hash sha256:e0d392bf5839d134ea88f44f715712
2a9928d640f59c3f8c48b52d85a5f9088c
[preflight] Running pre-flight checks
[preflight] Reading configuration from the cluster...
[preflight] FYI: You can look at this config file with 'kubectl -n kube-syst
em get cm kubeadm-config -o yaml'
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/conf
ig.yaml"
[kubelet-start] Writing kubelet environment file with flags to file "/var/li
b/kubelet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-check] Waiting for a healthy kubelet at http://127.0.0.1:10248/heal
thz. This can take up to 4m0s
[kubelet-check] The kubelet is healthy after 1.009694977s
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap

This node has joined the cluster:
* Certificate signing request was sent to apiserver and a response was recei
ved.
* The Kubelet was informed of the new secure connection details.

Run 'kubectl get nodes' on the control-plane to see this node join the clust
er.

khoavo@worker-node-2:~$
```

Bước 2.5: Kiểm tra hạ tầng

Đứng tại máy **Master Node**, gõ:

kubectl get nodes

Đợi cả 3 máy (`master-node`, `worker-node-1`, `worker-node-2`) hiển thị cột `STATUS` là `**Ready**`.

```
khoavo@master-node:~$ kubectl get nodes
NAME            STATUS     ROLES           AGE     VERSION
master-node     Ready      control-plane   4h41m   v1.30.14
worker-node-1   NotReady   <none>          44s     v1.30.14
worker-node-2   NotReady   <none>          20s     v1.30.14
khoavo@master-node:~$
```

📦 4. GIAI ĐOẠN 3: Triển Khai Ứng Dụng Pharmacy

Bước 3.1: Tạo thư mục dữ liệu MongoDB trên `worker-node-1`

Đăng nhập vào máy ảo `**worker-node-1**` và chạy:

```
sudo mkdir -p /mnt/data/mongo
sudo chmod 777 /mnt/data/mongo
```

```
khoavo@worker-node-1:~$ sudo mkdir -p /mnt/data/mongo
sudo chmod 777 /mnt/data/mongo
khoavo@worker-node-1:~$
```

Bước 3.2: Tạo 4 File Cấu hình YAML trên máy `master-node`

1. File `k8s-config.yaml`

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: pharmacy-config
data:
  MONGODB_DATABASE: "pharmacy"
  SPRING_PROFILES_ACTIVE: "prod"
---
apiVersion: v1
kind: Secret
metadata:
  name: pharmacy-secrets
type: Opaque
stringData:
```

```
  db-password: "SuperSecretPassword123"
  jwt-secret: "MySuperLongAndSecureJWTTokenKey1234567890"
  mongo-uri:
"mongodb://root:SuperSecretPassword123@mongodb:27017/pharmacy?authSource=admin"
```

2. File `k8s-mongodb.yaml`

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: mongo-pv
spec:
  capacity:
    storage: 10Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  hostPath:
    path: /mnt/data/mongo
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mongo-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
---
apiVersion: v1
kind: Service
metadata:
  name: mongodb
spec:
  ports:
    - port: 27017
      targetPort: 27017
  selector:
    app: mongodb
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mongodb
spec:
```

```
  replicas: 1
  selector:
    matchLabels:
      app: mongodb
  template:
    metadata:
      labels:
        app: mongodb
    spec:
      nodeSelector:
        kubernetes.io/hostname: worker-node-1
      containers:
        - name: mongodb
          image: mongo:6.0
          ports:
            - containerPort: 27017
          env:
            - name: MONGO_INITDB_ROOT_USERNAME
              value: "root"
            - name: MONGO_INITDB_ROOT_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: pharmacy-secrets
                  key: db-password
          resources:
            limits:
              cpu: "1.0"
              memory: 1Gi
            requests:
              cpu: "0.3"
              memory: 256Mi
          volumeMounts:
            - name: mongo-storage
              mountPath: /data/db
      volumes:
        - name: mongo-storage
          persistentVolumeClaim:
            claimName: mongo-pvc
```

3. File `k8s-backend.yaml`

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: spring-boot
spec:
  replicas: 2
```

```
selector:
  matchLabels:
    app: spring-boot
template:
  metadata:
    labels:
      app: spring-boot
  spec:
    affinity:
      podAntiAffinity:
        preferredDuringSchedulingIgnoredDuringExecution:
          - weight: 100
            podAffinityTerm:
              labelSelector:
                matchExpressions:
                  - key: app
                    operator: In
                    values:
                      - spring-boot
              topologyKey: "kubernetes.io/hostname"
    containers:
      - name: spring-boot
        image: dinhsi1401/pharmacy:2.0
        ports:
          - containerPort: 8081
        env:
          - name: MONGODB_DATABASE
            valueFrom:
              configMapKeyRef:
                name: pharmacy-config
                key: MONGODB_DATABASE
          - name: SPRING_PROFILES_ACTIVE
            valueFrom:
              configMapKeyRef:
                name: pharmacy-config
                key: SPRING_PROFILES_ACTIVE
          - name: SPRING_DATA_MONGODB_URI
            valueFrom:
              secretKeyRef:
                name: pharmacy-secrets
                key: mongo-uri
          - name: JWT_SECRET
            valueFrom:
              secretKeyRef:
                name: pharmacy-secrets
                key: jwt-secret
        resources:
          limits:
            cpu: "1.0"
            memory: 512Mi
```

```
          requests:
            cpu: "0.3"
            memory: 256Mi
          readinessProbe:
            tcpSocket:
              port: 8081
            initialDelaySeconds: 15
            periodSeconds: 10
          livenessProbe:
            tcpSocket:
              port: 8081
            initialDelaySeconds: 20
            periodSeconds: 20
---
apiVersion: v1
kind: Service
metadata:
  name: spring-boot
spec:
  ports:
    - port: 8081
      targetPort: 8081
  selector:
    app: spring-boot
```

4. File `k8s-frontend.yaml`

```
  apiVersion: apps/v1
  kind: Deployment
  metadata:
    name: frontend
  spec:
    replicas: 2
    selector:
      matchLabels:
        app: frontend
    template:
      metadata:
        labels:
          app: frontend
      spec:
        affinity:
          podAntiAffinity:
            preferredDuringSchedulingIgnoredDuringExecution:
              - weight: 100
                podAffinityTerm:
                  labelSelector:
```

```
              matchExpressions:
                - key: app
                  operator: In
                  values:
                    - frontend
            topologyKey: "kubernetes.io/hostname"
      containers:
        - name: frontend
          image: vokhoaecho/pharmacy-frontend:2.0
          ports:
            - containerPort: 80
          resources:
            limits:
              cpu: "0.5"
              memory: 128Mi
            requests:
              cpu: "0.1"
              memory: 64Mi
          readinessProbe:
            httpGet:
              path: /
              port: 80
            initialDelaySeconds: 5
            periodSeconds: 5
---
apiVersion: v1
kind: Service
metadata:
  name: frontend
spec:
  type: NodePort
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
  selector:
    app: frontend
```

```
khoavo@master-node:~$ nano k8s-config.yaml
khoavo@master-node:~$ nano k8s-mongodb.yaml
khoavo@master-node:~$ nano k8s-backend.yaml
khoavo@master-node:~$ nano k8s-frontend.yaml
khoavo@master-node:~$
```

Bước 3.3: Apply các fileManifest trên `master-node`

```
kubectl apply -f k8s-config.yaml
kubectl apply -f k8s-mongodb.yaml
kubectl apply -f k8s-backend.yaml
```

kubectl apply -f k8s-frontend.yaml

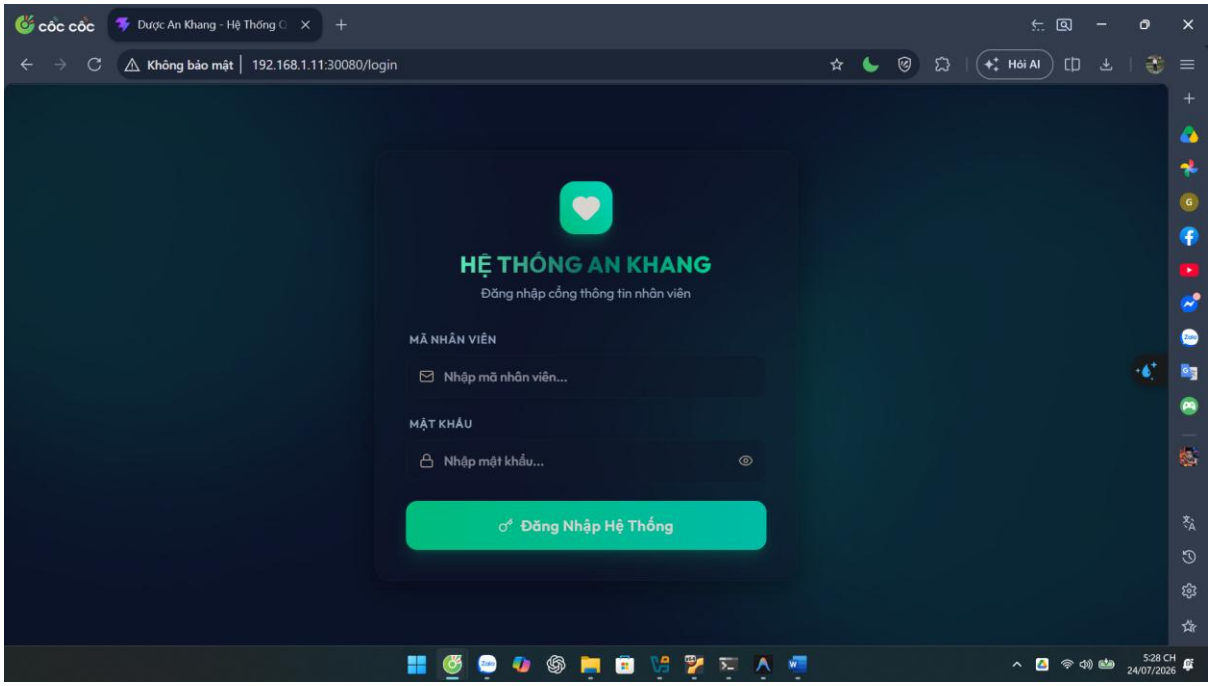
```
khoavo@master-node:~$ kubectl apply -f k8s-config.yaml
kubectl apply -f k8s-mongodb.yaml
kubectl apply -f k8s-backend.yaml
kubectl apply -f k8s-frontend.yaml
configmap/pharmacy-config created
secret/pharmacy-secrets created
persistentvolume/mongo-pv created
persistentvolumeclaim/mongo-pvc created
service/mongodb created
deployment.apps/mongodb created
deployment.apps/spring-boot created
service/spring-boot created
deployment.apps/frontend created
service/frontend created
khoavo@master-node:~$
```

🌐 5. Kiểm Tra & Truy Cập Ứng Dụng

1. Kiểm tra các Pod đã ở trạng thái `**Running**`:kubectl get pods -o wide

```
khoavo@master-node:~$ kubectl get pods -o wide
NAME                          READY   STATUS    RESTARTS   AGE     IP           NODE            NOMINATED NODE   READINESS GATES
frontend-6c96f58f98-42gl8     1/1     Running   0          143m    10.244.1.2   worker-node-1   <none>           <none>
frontend-6c96f58f98-ctbpk     1/1     Running   0          143m    10.244.1.4   worker-node-1   <none>           <none>
mongodb-85c575dc67-dvc9v      1/1     Running   0          143m    10.244.1.5   worker-node-1   <none>           <none>
spring-boot-649d77c77d-77zl4  1/1     Running   0          2m54s   10.244.1.7   worker-node-1   <none>           <none>
spring-boot-649d77c77d-krxn8  1/1     Running   0          3m15s   10.244.2.3   worker-node-2   <none>           <none>
khoavo@master-node:~$
```

2. Mở trình duyệt trên máy Windows thật và truy cập:
   - `**http://192.168.1.11:30080**` (IP Worker Node 1)
   - hoặc `**http://192.168.1.12:30080**` (IP Worker Node 2)



🛠 6. Xử Lý Các Sự Cố Thường Gặp (Troubleshooting)

1. Lỗi `Port 10250 is in use` hoặc `/var/lib/etcd is not empty` khi `init` / `join`:

- **Nguyên nhân**: Máy ảo hoặc node đã từng khởi tạo K8s trước đó (hoặc do clone máy ảo), khiến cổng 10250 bị chiếm và dữ liệu etcd cũ chưa dọn dẹp.
- **Cách xử lý** (Chạy trên máy bị lỗi):# Reset cấu hình Kubernetes
  sudo kubeadm reset -f

  # Dọn dẹp dữ liệu etcd, mạng CNI và config cũ
  sudo rm -rf /var/lib/etcd /etc/cni/net.d $HOME/.kube

  # Khởi động lại containerd
  sudo systemctl restart containerd

2. Lỗi Worker Node báo `NotReady` (`cni plugin not initialized`):

- **Nguyên nhân**: Plugin mạng Flannel CNI chưa được cài đặt hoặc Pod Flannel đang kẹt ở bước tải image (`Init:1/2`).
- **Cách xử lý**:

1. Trên máy **Master Node**, kiểm tra các Pod mạng Flannel:kubectl get pods -n kube-flannel

2. Nếu chưa áp dụng mạng Flannel, chạy:kubectl apply -f https://raw.githubusercontent.com/flannel-io/flannel/master/Documentation/kube-flannel.yml

3. Nếu Pod Flannel bị kẹt tải lâu, trên các máy **Worker Node** chạy:sudo mkdir -p /opt/cni/bin
sudo systemctl restart containerd

3. Lỗi các Pod ứng dụng ở trạng thái `Pending`:

- **Nguyên nhân**: Các máy Worker Node chưa join vào cụm hoặc đang ở trạng thái `NotReady`.
- **Cách xử lý**: Kiểm tra trạng thái node bằng `kubectl get nodes`. Đảm bảo tất cả các Worker Node hiển thị `**Ready**` thì Pod mới được xếp lịch và chuyển sang `ContainerCreating` -> `Running`.

4. Lỗi Pod bị `Pending` do `disk-pressure` (Ổ đĩa máy ảo bị đầy > 85%):

- **Nguyên nhân**: Bộ nhớ tạm của containerd phình to chiếm hết dung lượng đĩa.
- **Cách xử lý**: Xóa sạch bộ nhớ tạm containerd phình to trên máy ảo bị lỗi:sudo systemctl stop kubelet containerd
  sudo rm -rf /var/lib/containerd/*
  sudo systemctl start containerd kubelet

5. Lỗi `kubectl` báo connection refused (`localhost:8080`):

- **Nguyên nhân**: Thiếu file cấu hình quản trị `admin.conf` cho user hiện tại.
- **Cách xử lý**: Chạy lại các lệnh cấp quyền:mkdir -p $HOME/.kube
  sudo cp -i /etc/kubernetes/admin.conf HOME/.kube/config
  sudo chown $(id -u):(id -g) $HOME/.kube/config

6. Lỗi Worker Node cũ từng join bị trùng cấu hình:

Cách xử lý: Dọn dẹp cấu hình K8s cũ trước khi join lại:sudo kubeadm reset